



@qa_insights

SYSTEM DESIGN

COMPLETE NOTES

PART 1



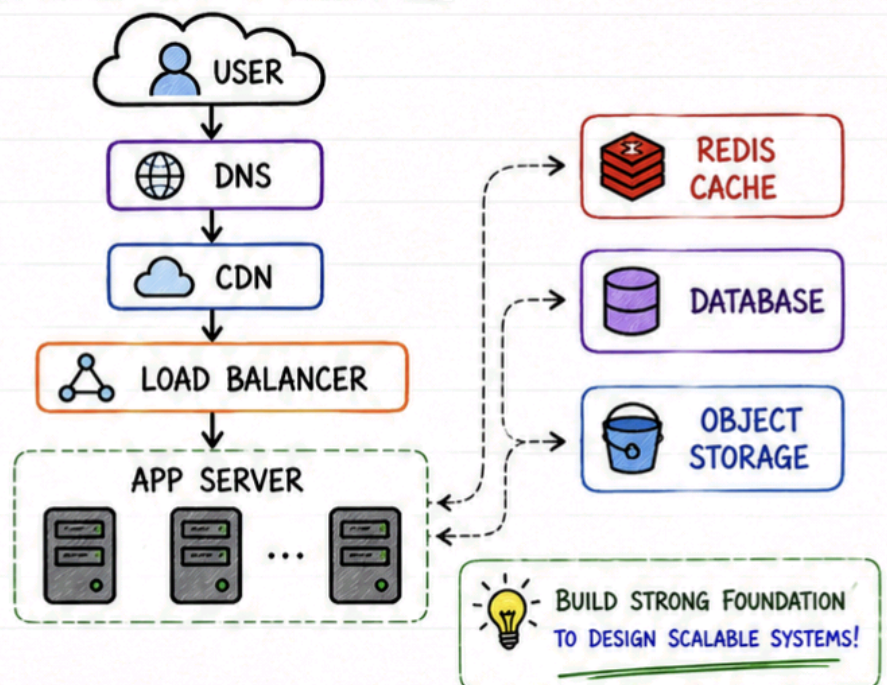
BEGINNER TO ADVANCED



- ✓ CORE CONCEPTS
- ✓ ARCHITECTURE BASICS
- ✓ SCALABILITY
- ✓ SYSTEM COMPONENTS
- ✓ REAL-WORLD EXAMPLES

WHAT YOU WILL LEARN?

- ✓ System Design Fundamentals
- ✓ High-Level Architecture
- ✓ Key Components Explained
- ✓ Scalability & Performance
- ✓ Best Practices
- ✓ Interview Tips



FOLLOW



SHARE



COMMENT



LIKE

What is System Design?

① Definition

System Design is the process of designing scalable, reliable, efficient and cost-effective software systems to meet business requirements.

Interview Tip

It's not about writing code.
It's about designing the right solution before you start coding.

② Why is System Design Important?

- ✓ Helps build **SCALABLE** systems
- ✓ Handles **HIGH TRAFFIC**
- ✓ Ensures **AVAILABILITY & RELIABILITY**
- ✓ Optimizes for **PERFORMANCE & COST**
- ✓ Prepares you for **SYSTEM DESIGN INTERVIEWS**

★ Remember

A good System Design balances **Scalability**, **Performance**, **Cost**, and **Maintainability**.

③ Real-world Examples



Instagram

- Millions of daily users
- Image & video uploads
- News feed & stories
- Scalable & Highly available



WhatsApp

- Real-time messaging
- End-to-end encryption
- Global scale
- Low latency & High availability



Amazon

- E-commerce platform
- Product search
- Order processing
- Highly available & Scalable

④ Goals of a Good System



Scalability - Handle growing users & data



Availability - System is always available



Performance - Low latency, High throughput

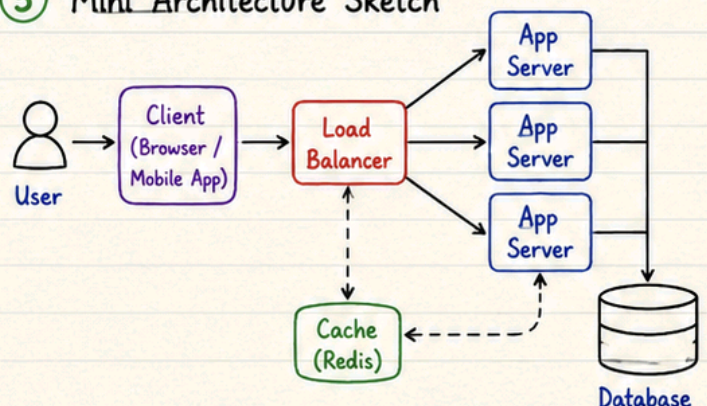


Reliability - Fault tolerant, Data safe



Cost Efficiency - Optimize infrastructure cost

⑤ Mini Architecture Sketch



System Design Terminology

Before diving deep, let's understand some important terms used in **System Design** interviews.



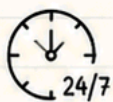
- ① **HLD (High-Level Design)** - Focus on system architecture. Shows big picture, components & their interactions.



- ② **LLD (Low-Level Design)** - Focus on component level. Covers classes, methods, data structures and algorithms.



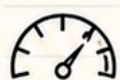
- ③ **Scalability** - Ability of system to handle growing traffic by adding resources.



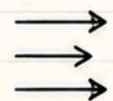
- ④ **Availability** - System is accessible and working when required. (Minimize downtime)



- ⑤ **Reliability** - System performs correctly without failures for a long time.



- ⑥ **Latency** - Time taken for a request to travel from client to server and back.



- ⑦ **Throughput** - Amount of work done by the system in a given time.



- ⑧ **Fault Tolerance** - System continues to operate even if some components fail.



Interview Tip

Always ask interviewer which of these are most important for their system. Every system has different priorities!

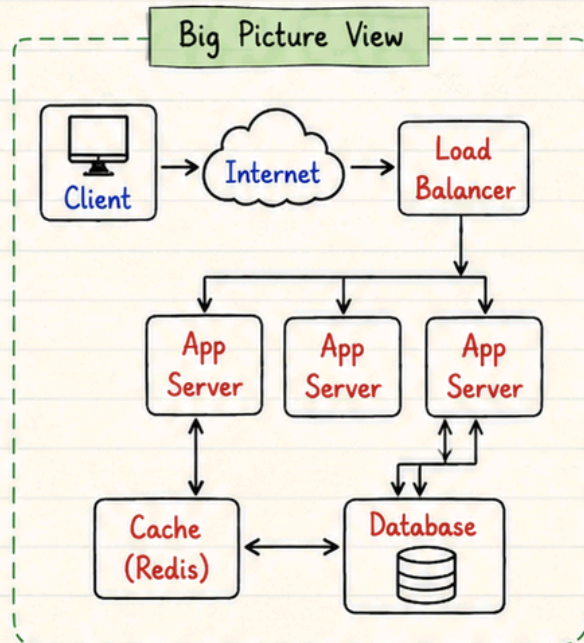


Why these terms matter?

- ✓ Help you think like an architect.
- ✓ Used in every **System Design** discussion.
- ✓ Helps in making trade-off decisions.
- ✓ Frequently asked in technical interviews.



Big Picture View



Functional vs Non-Functional Requirements

Before designing any system, we must understand **WHAT** the system should do and **HOW** well it should work.

① Functional Requirements



Define the specific features and behaviors of the system.

- ✓ What the system should do
- ✓ User visible features
- ✓ Business functionalities
- ✓ Core of the application

Example: User Registration, Login, Search, Add to Cart, Place Order

② Non-Functional Requirements



Define the quality attributes and constraints of the system.

- ✓ How well the system should work
- ✓ Performance, Scalability, Security
- ✓ Reliability, Availability, Usability
- ✓ Important for system success

Example: System should handle 10K users, 99.99% uptime, Response < 200ms

③ Key Differences

Aspect	Functional Requirements	Non-Functional Requirements
Focus	What the system does	How well the system works
Visibility	Visible to end users	Usually hidden from users
Examples	Login, Search, Checkout	Scalability, Security, Performance
Change Frequency	Changes occasionally	Changes frequently as system grows
Measurement	Easy to measure	Sometimes hard to measure
Importance	Critical for basic functionality	Critical for system success

④ Examples (E-commerce)

Functional

- User can register & login
- User can search products
- User can add to cart
- User can place order
- User can make payment

Non-Functional

- System should support 1M users
- Response time < 200ms
- 99.99% system availability
- Data should be secure
- System should be scalable

⑤ Interview Questions



- What are functional & non-functional requirements?
- Why are non-functional requirements important?
- How do you prioritize requirements?
- Give examples of both.
- How do requirements change with scale?



Interview Tip

Always start by clarifying requirements with the interviewer. It shows your communication & analysis skills!

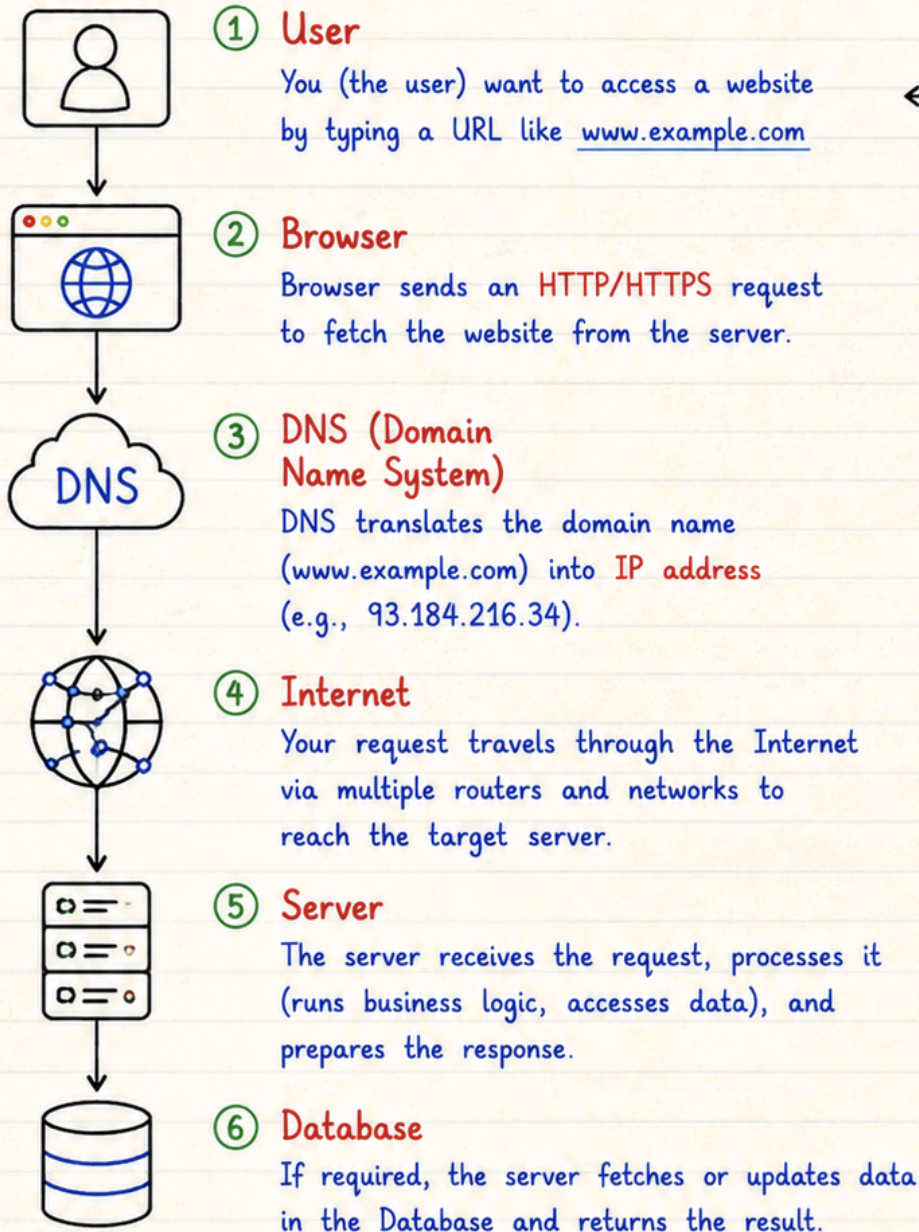


Common Mistake

Jumping into design without understanding requirements leads to **wrong architecture!**

How the Internet Works

Let's understand the journey of a request from you to a website.



Response

The response travels back the same path (Server → Internet → DNS → Browser → User) and the website is displayed in your browser.



Key Takeaway

The Internet is a network of networks that helps your request reach the right server, and brings the response back to you.

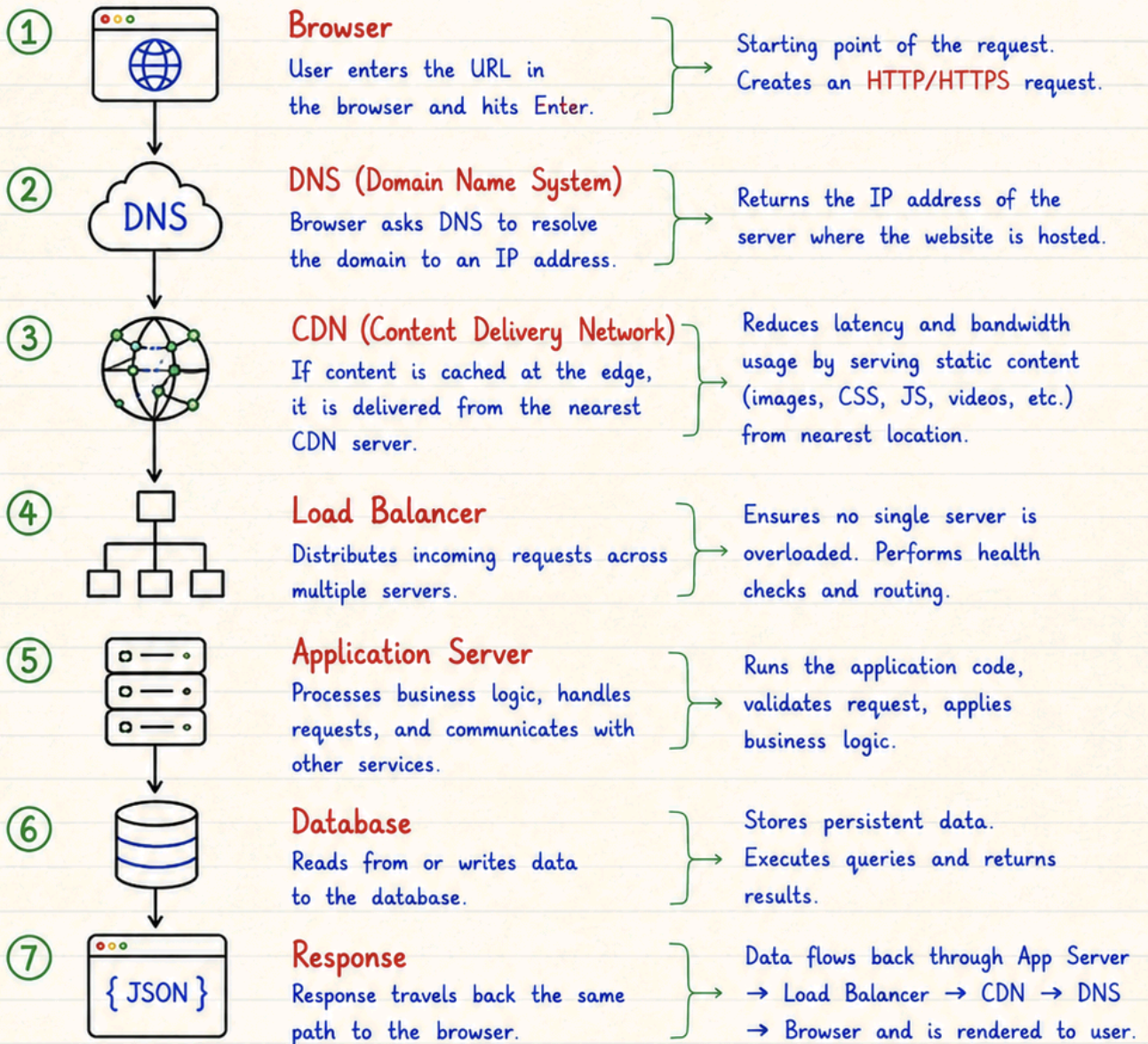
Interview Tip



Be ready to explain this flow clearly in System Design interviews!

Complete Request Lifecycle

Let's see the end-to-end flow of a request when a user tries to access a website or application.



Key Takeaway

- ✓ Each component plays a specific role.
- ✓ The flow ensures fast, reliable & scalable delivery.
- ✓ Bottlenecks can occur at any layer.

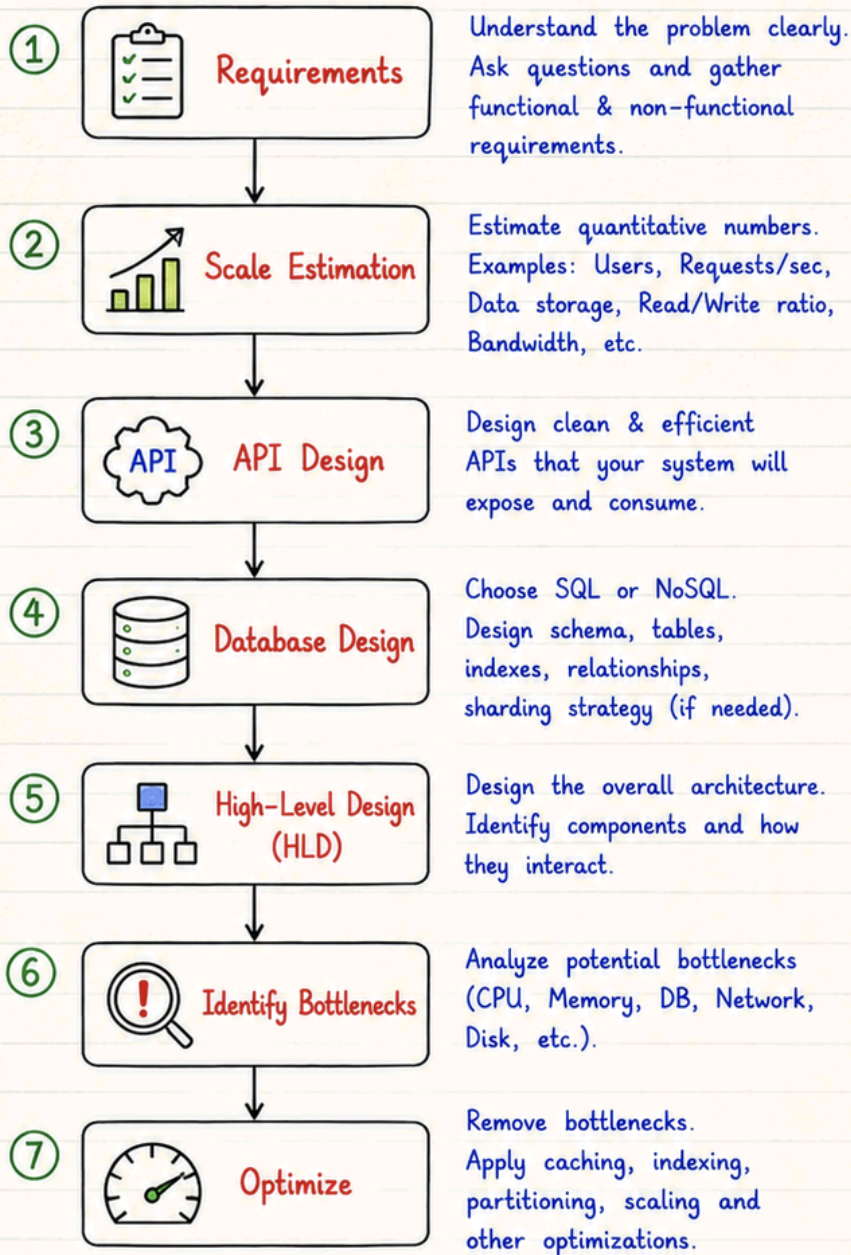


Interview Tip

Always explain the **request flow** clearly in **System Design** interviews. Start high-level and dive deeper if asked.

System Design Interview Workflow

Follow this structured approach to design scalable, reliable and efficient systems in interviews.



★ Key Points

- ✓ Start from **requirements**, not technology.
- ✓ Always make **assumptions** and communicate.
- ✓ Keep the design **simple** and scalable.
- ✓ Discuss **trade-offs** clearly.
- ✓ Iterate and **improve** the design.

💡 Example Assumptions

- 1 Million users
- 10K requests/sec
- 70% Read, 30% Write
- Data retention: 5 years
- Average response time < 200ms
- 99.9% Availability

🎯 Interview Tip

Think out loud, draw diagrams, ask clarifying questions and explain your **thought process** at every step.

That's what interviewers really **evaluate**!

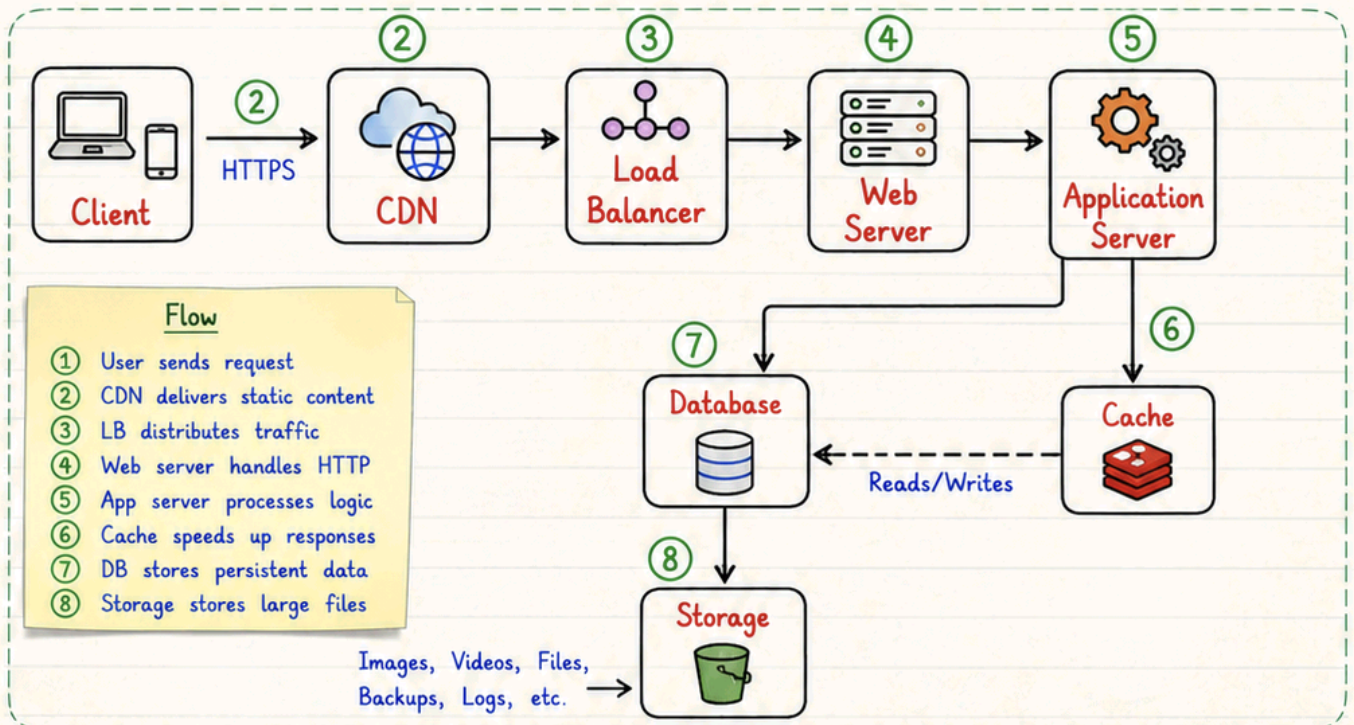


Remember There is no single "right" design. The goal is to communicate, justify trade-offs and design a system that meets requirements.



Core Components of a System

Every scalable system is built using a set of core components that work together to handle user requests efficiently.



- 1 Client** - Browser / Mobile App / Third-party Service that initiates a request.
- 2 CDN** - Caches and delivers static content from edge locations to reduce latency.
- 3 Load Balancer** - Distributes incoming requests across multiple servers to ensure high availability.
- 4 Web Server** - Handles HTTP/HTTPS requests, serves static content, and forwards dynamic requests.
- 5 Application Server** - Contains business logic, processes requests, and communicates with other services.
- 6 Cache** - Stores frequently accessed data in-memory for fast retrieval (e.g., Redis, Memcached).
- 7 Database** - Stores structured data permanently (e.g., MySQL, PostgreSQL, MongoDB).
- 8 Storage** - Stores unstructured data like files, media, backups using object storage (e.g., AWS S3).

Why These Components? ★

- ✓ Separation of concerns
- ✓ Improves Scalability
- ✓ Enhances Availability
- ✓ Boosts Performance
- ✓ Easy to maintain & expand

Remember 💡

Not every system needs all components. Choose based on requirements, scale & cost.

Interview Tip 🎯

Always start HLD by identifying these components and how they interact with each other.

API Design Fundamentals

Good API design makes your system easy to use, scalable, secure and developer friendly.

① REST



- Architectural style for building web services.
- Stateless, client-server communication.
- Uses standard HTTP methods.

② HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Create a new resource
PUT	Update an existing resource
PATCH	Partially update a resource
DELETE	Delete a resource

③ JSON



- Lightweight data-interchange format.
- Easy for humans to read and write.
- Most APIs use JSON in request & response body.

Example:

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john@example.com"
}
```

④ Status Codes

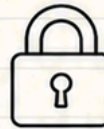
1XX	Informational	- Request received
2XX	Success	- Request successful
3XX	Redirection	- Further action needed
4XX	Client Error	- Invalid request
5XX	Server Error	- Server failed to process

Interview Tip

Always discuss API design early in the interview. Show clear endpoints, request/response format, status codes and versioning strategy.



⑤ Authentication



- Verifies the identity of the user.
- Common methods: API Key, Basic Auth, OAuth2, JWT.
- Protects APIs from unauthorized access.

Example (JWT): Bearer eyJhbGciOiJIUzI1NiIs...

⑥ Pagination



- Helps in handling large datasets.
- Prevents huge payloads.
- Common params: page, size, limit, offset, cursor.

Example: GET /users?page=2&size=20

⑦ Versioning



- Manage API changes without breaking existing clients.
- Common approaches: /v1/users, Header (v1), Query Param (version=1)
- Keep older versions for backward compatibility.



Best Practices

- ✓ Use nouns, not verbs in URLs.
- ✓ Keep APIs simple and consistent.
- ✓ Use proper status codes.
- ✓ Implement rate limiting.
- ✓ Secure your APIs.
- ✓ Document your APIs (Swagger / OpenAPI).

Database Fundamentals

Databases store, organize and manage data efficiently.

Choosing the right database is critical to system design.

① SQL (Relational Database)

- Data is stored in tables (rows & columns).
- Schema is predefined.
- Relationships using keys.
- Examples: MySQL, PostgreSQL, Oracle.

Users Table

id (PK)	name	email	age
1	John	john@example.com	28
2	Jane	jane@example.com	25
3	Mike	mike@example.com	30

PK → Primary Key (unique identifier)

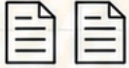
② NoSQL (Non-Relational Database)

- Schema-less / flexible schema.
- Stores data in documents, key-value, wide columns or graphs.
- Horizontal scaling is easier.
- Examples: MongoDB, Cassandra, DynamoDB.

Example (Document in MongoDB)

```
{
  "_id": ObjectId("64ab..."),
  "name": "John",
  "email": "john@example.com",
  "age": 28,
  "roles": ["user", "admin"]
}
```

③ Tables vs Collections

SQL (Table)	NoSQL (Collection)
	
Fixed schema	Flexible schema
Rows & Columns	Documents (JSON/BSON)
Joins (using keys)	Embedded documents / References
ACID Transactions	BASE (Eventual Consistency)
Examples: MySQL, PostgreSQL	Examples: MongoDB, Cassandra, DynamoDB

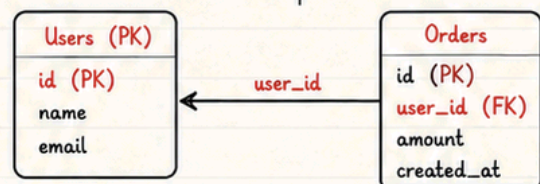
④ Primary Key vs Foreign Key

Primary Key (PK)

- Uniquely identifies a record.
- Cannot be null.
- Only one PK per table.

Foreign Key (FK)

- References the PK of another table.
- Can have duplicate values.
- Maintains relationships.



⑤ ACID (SQL Databases)

- ✓ A - Atomicity: All or nothing.
- ✓ C - Consistency: Data remains valid.
- ✓ I - Isolation: Transactions don't interfere.
- ✓ D - Durability: Data is permanent.

⑥ BASE (NoSQL Databases)

- ✓ B - Basically Available
- ✓ A - Soft State
- ✓ S - Eventually Consistent
- ✓ E - Designed for high availability and scalability.



Key Takeaway

Use SQL when you need strong consistency and complex relationships.

Use NoSQL when you need high scalability, flexibility and massive data handling.

🎯 Interview Tip

Be ready to explain when to choose SQL vs NoSQL, the trade-offs, and how you will design schema for both.



Common Mistake

Choosing NoSQL just for scale, without understanding consistency requirements. Always match the DB to the use case.

Examples

E-commerce, Banking → SQL
Social Feed, Messaging, IoT → NoSQL

Database Indexing

Indexing is a technique used to speed up data retrieval operations on a database table.

① What is an Index?

- An index is a special data structure that stores a small portion of the table data in an easy-to-traverse format.
- It works like the index of a book - helps find data quickly without scanning the entire table.

② Why Use Index?

- Speeds up SELECT queries.
- Helps in filtering, sorting, grouping and joining data.
- Improves overall database performance.

③ Advantages

- ✓ Faster data retrieval (reduced query time).
- ✓ Improves performance of complex queries.
- ✓ Helps in enforcing uniqueness (Unique Index).
- ✓ Speeds up JOIN operations.

④ Drawbacks

- ✗ Consumes additional storage space.
- ✗ Slows down INSERT, UPDATE, DELETE operations (index needs to be updated).
- ✗ Too many indexes can degrade performance.



Key Takeaway

Use indexes on columns that are frequently searched, filtered, or used in joins - but avoid over-indexing!



Interview Tip

Always discuss which columns you will index and why during system design. Mention the trade-offs clearly.

⑤ How Index Works?

Index on name column

Table (Users)

Index (B-Tree)

Alice
Bob
Carol
David
...

id (PK)	name	email	age
1	Alice	alice@mail.com	25
2	Bob	bob@mail.com	30
3	Carol	carol@mail.com	28
4	David	david@mail.com	35
...			

Instead of scanning the whole table, DB uses the index to quickly locate the matching rows.

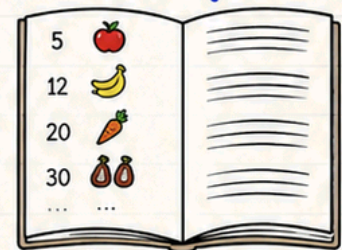
⑥ Real-world Analogy

Think of a book:

Index (Book)

Apple		5
Banana		12
Carrot		20
Dates		30
...		

Actual Pages



You don't read the whole book to find "Carrot", you go to the index and jump to page 20!

⑦ Visual Example

Without Index (Full Table Scan)

Row 1
Row 2
Row 3
Row 4
...
Row N

Scan every row to find data

With Index

Index

Key A
Key B
Key C
...
Key Z

Table

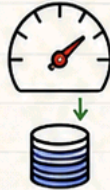
Jump directly to matching rows

Caching in System Design

Caching stores frequently accessed data in a fast storage layer so that we can reduce latency, database load and improve performance.

① Why Caching?

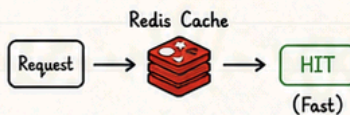
- ✓ Reduces response time (lower latency)
- ✓ Decreases load on database
- ✓ Improves system throughput
- ✓ Helps in handling high traffic
- ✓ Cost-effective (fewer DB calls)



② Cache Basics

• Cache Hit

Data is found in cache.



• Cache Miss

Data is not in cache.



⑥ Cache-Aside Flow (Steps)

- ① User sends request to application.
- ② App checks cache for the data.
 - If HIT → return data.
 - If MISS → go to DB.
- ③ Fetch data from database.
- ④ Return data to user.
- ⑤ Store the data in cache with TTL.
- ⑥ Next time → data will be served from cache (HIT).



🎯 Interview Tip

- Explain why caching improves performance.
- Discuss trade-offs (consistency vs performance).
- Know when NOT to use cache (write-heavy data, real-time critical data).

③ Redis (In-Memory Cache)

- Open source in-memory data structure store.
- Supports strings, hashes, lists, sets, sorted sets.
- Extremely fast (microseconds).
- Used widely for caching, sessions, rate limiting, leaderboards, counters, etc.



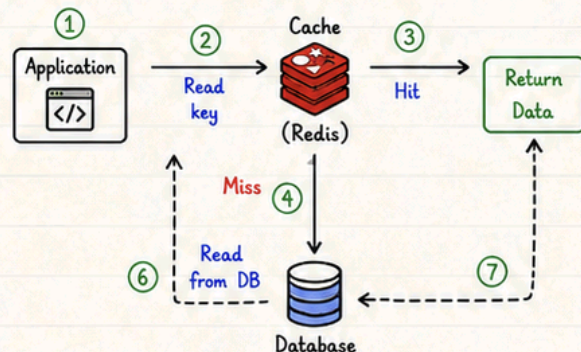
④ TTL (Time To Live)

- TTL defines how long a key stays in cache.
- After TTL expires, key is removed automatically.



Key: user:123
TTL: 3600s (1 hour)

⑤ Cache-Aside Pattern (Most Common)



⑦ When to Use Cache?

- ✓ Read-heavy workloads
- ✓ Slow database
- ✓ Expensive computations
- ✓ Rate limiting
- ✓ Session management
- ✓ Leaderboards, Counters, Feeds



Key Takeaway

Cache makes your system **FAST**, but always handle data consistency and invalidation carefully.

Pros vs Cons



Advantages

- Faster responses
- Reduces DB load
- Improves scalability
- Better user experience



Drawbacks

- Data can become stale (inconsistency)
- Extra memory usage
- Complex cache invalidation
- Adds another component to manage

Popular Use Cases

- User sessions
- Product catalog
- Feed (posts, timelines)
- Shopping cart
- API rate limiting

Content Delivery Network (CDN)

A CDN is a distributed network of servers that deliver static content to users from the nearest location, reducing latency and improving performance.

① What is CDN?

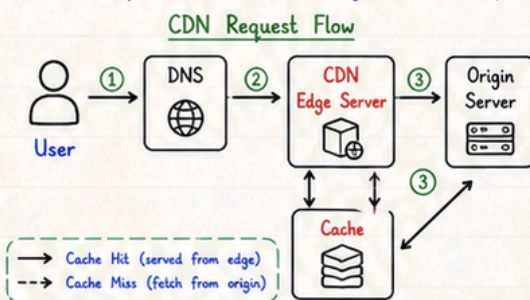
- CDN (Content Delivery Network) is a globally distributed system of edge servers.
- It caches and delivers content closer to users to reduce latency and bandwidth usage on origin servers.
- Works at the network edge.

② Static vs Dynamic Content

Static Content	Dynamic Content
• Does not change frequently. • Can be cached at edge. • Examples: Images, CSS, JS, Videos, Fonts, PDFs	• Changes frequently per user/request. • Usually not cached at edge (or short TTL). • Examples: User Dashboard, API responses, Search results

③ How CDN Works?

- ① User requests content from the website.
- ② DNS resolves the domain to the nearest CDN edge.
- ③ Edge server checks if the content is cached.
 - If HIT → content is served from edge.
 - If MISS → edge fetches from origin server, caches it and delivers to user.
- ④ Future requests are served from edge (until TTL expires).



Key Takeaway

CDN is essential for fast, scalable and highly available systems, especially for global users.

④ Global Edge Servers



- CDN has edge servers in multiple regions (PoPs).
- Users are served from the closest edge → lower latency.
- If content is not in edge → fetched from origin.

⑤ Benefits of CDN

- ✓ Reduces Latency (content delivered closer to user)
- ✓ Improves Website Performance
- ✓ Reduces Load on Origin Server
- ✓ Saves Bandwidth & Costs
- ✓ Handles Traffic Spikes (High Scalability)
- ✓ Improves Availability & Reliability
- ✓ DDoS Protection (absorbs malicious traffic)



⑥ Popular CDN Providers



⑦ Real-world Examples

- Netflix - Streams movies via CDN to millions globally.
- YouTube - Delivers videos from nearest edge servers.
- Amazon - Uses CloudFront to serve product images, JS, CSS and other assets.
- Microsoft - Uses Azure CDN for docs, updates, media.



Interview Tip

When asked about CDN in interviews, explain: purpose, how it works, edge servers, cache hit/miss, benefits and real-world use.



Remember

CDN is great for static content. Dynamic content can be cached with short TTL or personalized caching.

Load Balancer

A Load Balancer distributes incoming network traffic across multiple servers to improve performance, reliability and scalability.

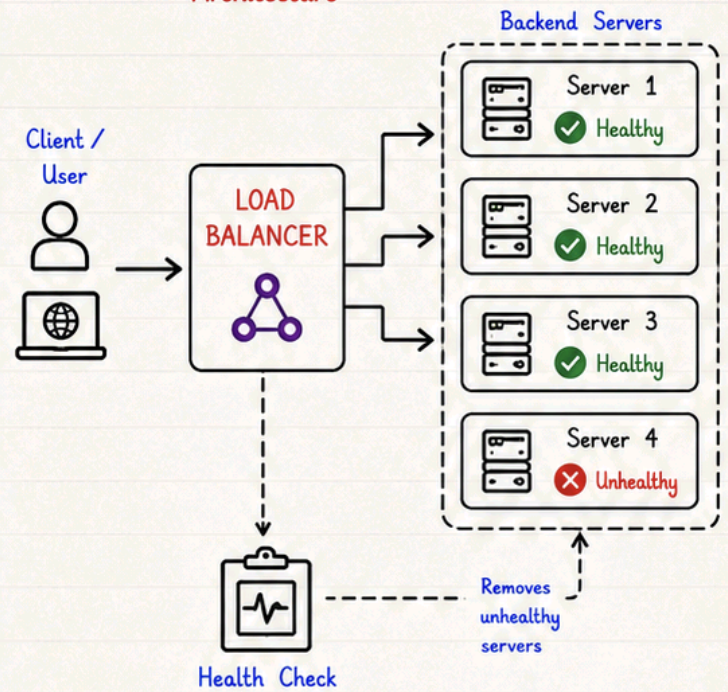
① Why do we need Load Balancer?

- ✓ Distributes traffic evenly across servers.
- ✓ Improves performance and response time.
- ✓ Prevents any single server from overloading.
- ✓ Provides High Availability & Fault Tolerance.
- ✓ Allows easy horizontal scaling.
- ✓ Performs Health Checks and removes unhealthy servers.

② How it Works?

- ① Client sends request.
- ② Load Balancer receives the request.
- ③ It selects a healthy server using a load balancing algorithm.
- ④ Request is forwarded to the server.
- ⑤ Server processes the request and sends response back.
- ⑥ Response is returned to the client via Load Balancer.

Architecture



③ Load Balancing Algorithms

Algorithm	How it Works
Round Robin	Requests are distributed sequentially to each server in a circular order.
Least Connections	Requests are sent to the server with the least active connections.
IP Hash	Uses client IP to always route to the same server.
Weighted Round Robin	Distributes requests based on server weights (capacity).

④ Health Checks

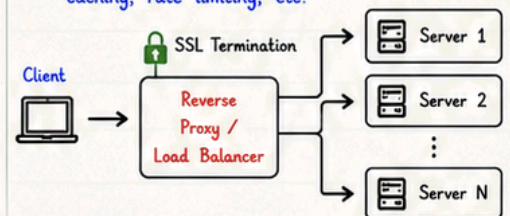
- Load Balancer periodically checks the health of backend servers.
- If a server doesn't respond or fails, it is marked as unhealthy.
- Traffic is not sent to unhealthy servers.
- When the server recovers, it is added back to the pool.

Common Checks:

- HTTP/HTTPS check
- TCP/Port check
- Custom application check

⑤ Reverse Proxy

- Load Balancer acts as a Reverse Proxy.
- It sits in front of backend servers.
- It hides the internal server details from clients.
- Handles SSL termination, compression, caching, rate limiting, etc.



⑥ Benefits

- ✓ High Availability
- ✓ Better Performance
- ✓ Scalability
- ✓ Fault Tolerance
- ✓ Easy Maintenance
- ✓ Zero Downtime Deployments
- ✓ Efficient Resource Utilization



Key Takeaway

Load Balancer is the traffic cop of your system - it ensures the right request goes to the right server at the right time!



Interview Tip

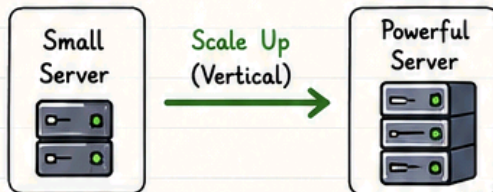
Always mention: why LB is needed, algorithms, health checks, and how it improves scalability & availability.

Scaling Applications

Scaling means handling more users, more requests and more data without compromising performance.

① Vertical Scaling (Scale Up)

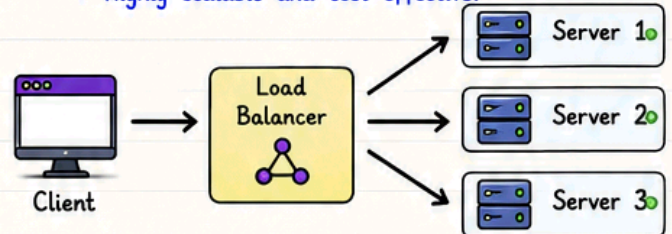
- Increase the power of a single server (CPU, RAM, Storage).
- Simple to implement but has limitations.



💡 **Best for:** Small to medium scale apps with predictable growth.

② Horizontal Scaling (Scale Out)

- Add more servers to handle the load.
- Highly scalable and cost-effective.



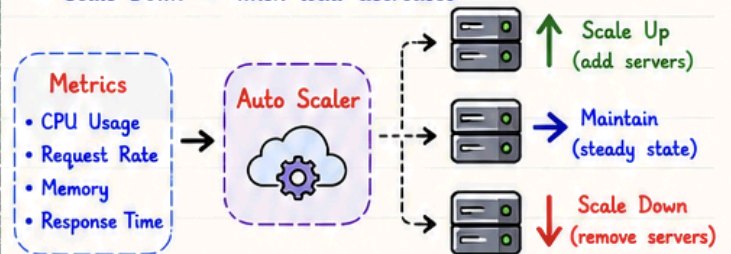
✅ **Best for:** Large scale apps with unpredictable or high traffic.

③ Stateless vs Stateful Servers

Stateless Server	Stateful Server
✓ Does not store client session/data. ✓ Each request is independent. ✓ Easy to scale horizontally. ✓ Ideal for Web APIs, Microservices. 	✗ Stores client session or data. ✗ Requests depend on previous state. ✗ Harder to scale horizontally. ✗ Example: Traditional Monolith apps. 

④ Auto Scaling

- Automatically adjusts the number of servers based on the load.
- Scale Up → when load increases
- Scale Down → when load decreases



⑤ Vertical Scaling vs Horizontal Scaling

Aspect	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
Definition	Increase capacity of a single server	Add more servers to the system
Example	More CPU, RAM, Bigger Server	Add more instances/servers
Cost	High (limited by hardware)	Cost-effective (pay as you grow)
Scalability	Limited (has upper limit)	Highly Scalable (almost unlimited)
Complexity	Simple to implement	More complex (requires LB, etc.)
Downtime Risk	High (single point of failure)	Low (distributes the load)
Best For	Small traffic, predictable load	Large traffic, high availability

⑥ When to Use?

- Use **Vertical Scaling** when:
 - Traffic is low/medium
 - Quick and simple solution
 - Budget allows powerful server
- Use **Horizontal Scaling** when:
 - Traffic is high and unpredictable
 - Need High Availability
 - Want to handle more users

⑦ Real-world Examples

 Netflix	→ Horizontal Scaling (using many servers, AWS)
 Instagram	→ Horizontal Scaling (Load Balancer + Auto Scaling)
 Small Blog	→ Vertical Scaling (Larger Server)

⑧ Key Takeaway

- ✓ Design **Stateless Services** for easy horizontal scaling.
- ✓ Use **Load Balancer + Auto Scaling** for high performance.
- ✓ Monitor metrics and scale **smartly**, not blindly!

⑨ Quick Tips / Interview Points

- ★ Always prefer **Horizontal Scaling** for long-term growth.
- ★ Keep servers **Stateless** to make scaling easier.
- ★ Use **Auto Scaling** to save cost and handle peak traffic.



Interview Tip: When asked about scaling, explain both Vertical and Horizontal scaling with pros, cons, use cases, and real-world examples.

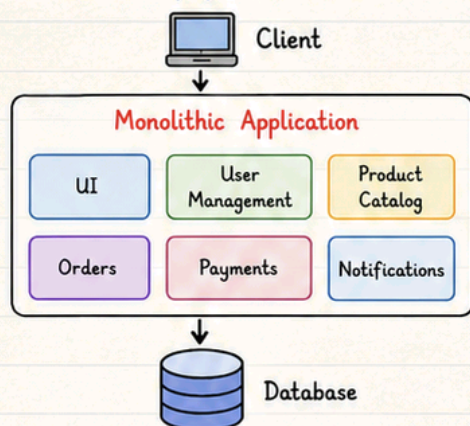


Monolithic vs Microservices Architecture

Both Monolithic and Microservices are valid architecture styles.
The choice depends on the size of the system, team, and business needs.

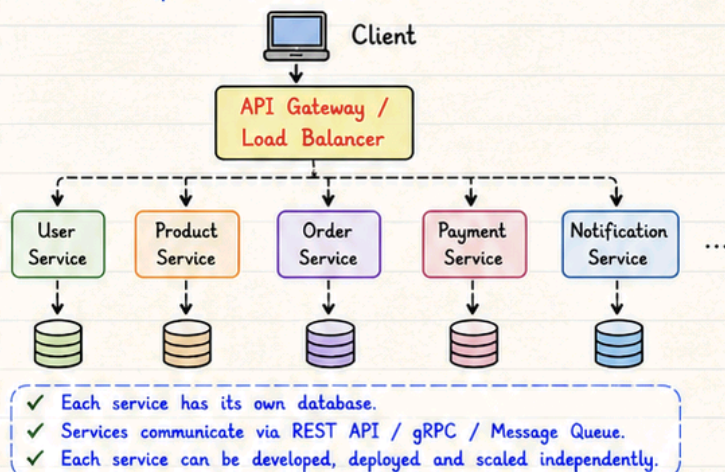
① Monolithic Architecture

- The entire application is built as a single unit and deployed as one artifact.



② Microservices Architecture

- The application is broken down into small, independent services that communicate via APIs.



③ Monolithic vs Microservices Comparison

Aspect	Monolithic	Microservices
Structure	Single codebase, single deployable	Multiple independent services
Deployment	Deploy the entire application	Deploy services independently
Scalability	Scale the whole application	Scale individual services
Technology	Single tech stack	Polyglot (different tech stacks)
Data Management	Single shared database	Database per service
Fault Isolation	Failure in one module can crash the whole app	Failure in one service doesn't affect others
Team Structure	One large team	Small independent teams
Complexity	Simple to start	More complex (network, deploy, monitoring, testing)
Performance	Good for small apps	Better for large, high traffic apps
Best For	Small apps, MVPs, simple systems	Large systems, complex domains, fast evolving products

④ Advantages & Disadvantages

Monolithic

Advantages

- ✓ Easy to develop and deploy
- ✓ Simple architecture
- ✓ Easier testing and debugging
- ✓ Good performance (no network calls)

Disadvantages

- ✗ Hard to scale
- ✗ Changes affect the whole system
- ✗ Longer deployment cycles
- ✗ Technology lock-in

Microservices

Advantages

- ✓ Highly scalable
- ✓ Independent deployment
- ✓ Technology flexibility
- ✓ Better fault isolation
- ✓ Faster development by teams

Disadvantages

- ✗ Complex architecture
- ✗ Network latency
- ✗ Data consistency challenges
- ✗ Harder testing & monitoring
- ✗ Higher operational cost

⑤ When to Use?

Use Monolithic When:

- ✓ You are building a small application or MVP
- ✓ The domain is simple and unlikely to change
- ✓ You have a small team
- ✓ You want to iterate quickly
- ✓ Operations resources are limited

Use Microservices When:

- ✓ The application is large and complex
- ✓ You expect rapid feature growth
- ✓ Different teams own different features
- ✓ You need high availability & scalability
- ✓ You need technology flexibility

⑥ Real-world Examples

Monolithic



Microservices



Key Takeaway

Monolith is **Simple** to build.

Microservices is **Powerful** to scale.

Choose based on your team, domain complexity, scalability needs and long-term goals.



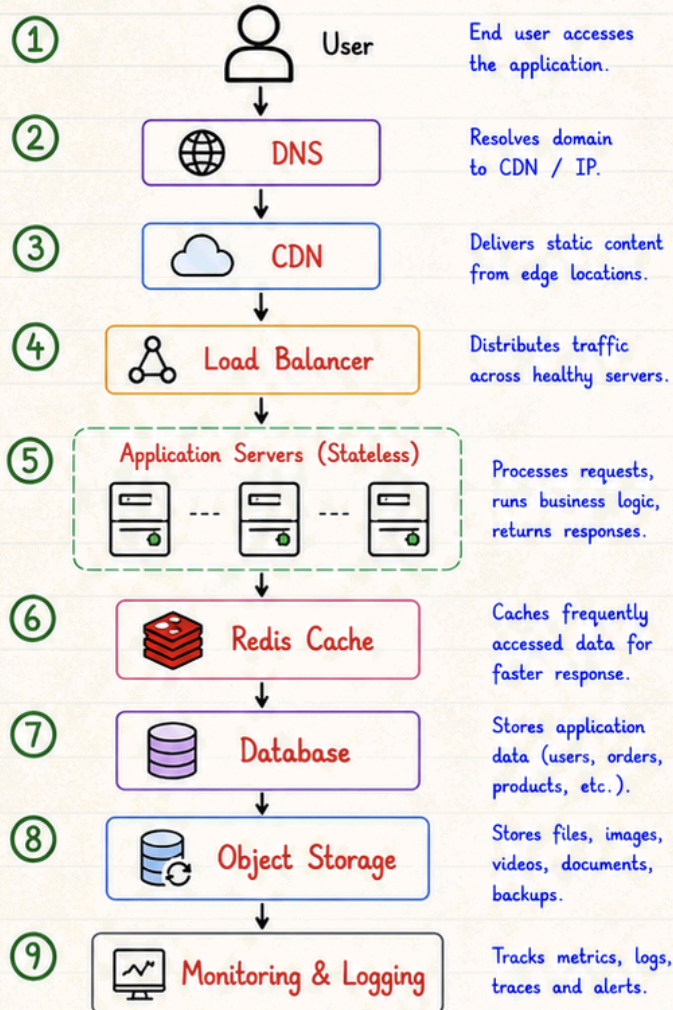
Interview Tip

In interviews, discuss trade-offs clearly.
There is no one-size-fits-all. Show that you can justify the right choice for the given scenario.



Putting Everything Together

A complete high-level architecture of a modern scalable web application.



Components Explained

- ① **User**
User sends a request via Browser / Mobile App.
- ② **DNS**
Maps domain name to the IP address of CDN / Load Balancer.
- ③ **CDN (Content Delivery Network)**
Serves static content (CSS, JS, Images, Videos) from nearest edge location to reduce latency.
- ④ **Load Balancer**
Distributes incoming requests to multiple Application Servers using algorithms (Round Robin, Least Connections, etc.).
- ⑤ **Application Servers**
Stateless servers that handle requests, execute business logic and interact with cache / database.
- ⑥ **Redis Cache**
In-memory data store used for caching sessions, tokens, frequently read data, rate limiting, leaderboards, etc.
- ⑦ **Database**
Primary data store for persistent data. Can be SQL or NoSQL with replication for High Availability.
- ⑧ **Object Storage**
Stores large files and media. Scalable, durable and accessible via URLs.
- ⑨ **Monitoring & Logging**
Collects metrics, logs and traces to monitor health, performance and errors. (e.g., Prometheus, Grafana, ELK, Datadog)

★ Key Benefits

- ✓ High Performance & Low Latency
- ✓ Scalable & Highly Available
- ✓ Fault Tolerant & Reliable
- ✓ Better User Experience
- ✓ Cost Effective with CDN & Caching



Key Takeaway

This is a common architecture used in most scalable systems.

Each layer has a specific role and together they ensure the system is **Fast, Scalable, Reliable & Maintainable**.



Interview Tip

When explaining architecture in interviews, always discuss:

- Flow of requests
- Role of each component
- Scalability & Reliability
- Trade-offs & Alternatives



Considerations

- Cache Invalidation Strategy
- Database Replication & Backup
- Data Consistency (CAP Theorem)
- Security (Auth, ssl, Rate Limiting)
- Disaster Recovery Plan
- Cost Optimization



Real-world Example

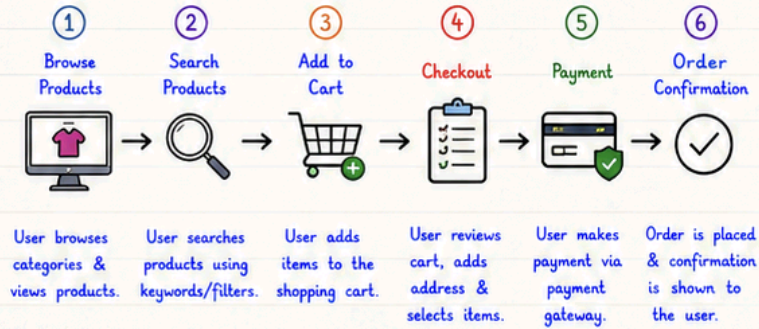
NETFLIX **amazon** **Uber** **Instagram**

All follow similar architecture with different optimizations.

Design a Simple E-Commerce System

Let's design a simple E-Commerce system and walk through the main user flow and high-level architecture.

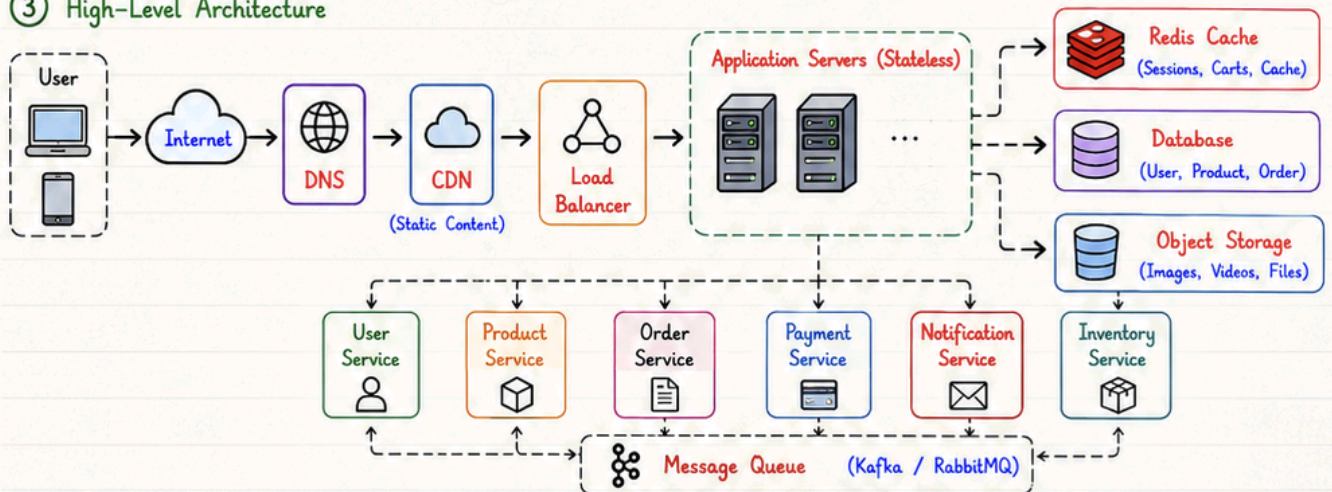
① User Flow (Happy Path)



② Key Functional Modules

- **Product Catalog** : Manage products, categories, inventory
- **Search Service** : Full-text search for products
- **Cart Service** : Manage user cart & items
- **Order Service** : Create & manage orders
- **Payment Service** : Handle payments via gateways
- **User Service** : Manage users, profiles, addresses
- **Notification Service** : Send emails/SMS for updates
- **Inventory Service** : Track stock & availability

③ High-Level Architecture



④ Walk Through (Step by Step)

1. User requests the website / app.
2. DNS resolves the domain to CDN / Load Balancer.
3. CDN serves static content (images, CSS, JS).
4. Request goes to Load Balancer which routes to healthy Application Server.
5. Application Server calls required Microservices.
6. Data is fetched from Redis Cache / Database.
7. For long tasks (email, invoice), services publish events to Message Queue.
8. Finally, user gets response & order confirmation.

⑤ Tech Stack (Example)

Frontend	React / Next.js / Flutter
Backend	Java / Node.js / Python
API Style	REST API / GraphQL
Database	MySQL / PostgreSQL
Cache	Redis
Search	Elasticsearch
Message Queue	Kafka / RabbitMQ
Storage	AWS S3 / GCS
Cloud	AWS / Azure / GCP
Container	Docker / Kubernetes

⑥ Non-Functional Requirements

- ✓ **Scalability** : Handle millions of users & requests.
- ✓ **Availability** : 99.9% uptime with redundancy.
- ✓ **Performance** : Low latency & high throughput.
- ✓ **Reliability** : Fault tolerant & data safe.
- ✓ **Security** : Secure payments, auth, data privacy.
- ✓ **Consistency** : Data consistency in transactions.
- ✓ **Monitoring** : Logs, metrics, alerts, tracing.

Interview Tip

Always discuss assumptions, scale, trade-offs, bottlenecks, and how you would improve the system for high scale.



Key Takeaway

Break the system into small services, choose the right components, and design for scale, reliability and performance from Day 1.

Real-world Examples

amazon Flipkart meesho

All these platforms follow similar architecture with their own optimizations.

Quick Revision Cheat Sheet

One-page summary of System Design essentials!

① Core Terms

- Scalability**: Handle more load by adding resources.
- Availability**: System is up and running when needed.
- Reliability**: System works correctly without failures.
- Latency**: Time taken to respond to a request.
- Throughput**: Number of requests handled per second.
- Stateless**: No session/data stored on server.
- Stateful**: Server stores session/data.
- CAP Theorem**: Consistency, Availability, Partition Tolerance.

② Interview Workflow

- Understand Requirements**
Clarify functional & non-functional requirements.
- High-Level Design (HLD)**
Identify major components & their interactions.
- Low-Level Design (LLD)**
Deep dive into components, APIs, data models.
- Data Design**
DB schema, indexing, relationships.
- Scalability & Reliability**
Caching, scaling, replication, fault tolerance.
- Trade-offs & Conclusion**
Discuss trade-offs, alternatives & final design.

③ Core Components

Client / Browser	Sends requests via HTTP/HTTPS	Cache : Stores frequently Transaction.
DNS	Resolves domain name to IP address	Database : Stores persistent data (SQL/NoSQL)
CDN	Delivers static content from edge	Object Storage : Stores files, images.
Load Balancer	Distributes traffic across servers	Message Queue : Async, communication (Kafka/RabbitMQ)
Web / App Server	Handles business logic & APIs	
Cache (Redis)	Stores frequently accessed data	
Database	Stores persistent data (SQL/NoSQL)	

④ SQL vs NoSQL

	SQL (Relational)	NoSQL (Non-Relational)
Structure	Table based	Flexible (Key-Value, Document, Column, Graph)
Schema	Fixed Schema	Dynamic Schema
Query	SQL	NoSQL Query / API
Scalability	Vertical Scaling	Horizontal Scaling
Consistency	Strong Consistency	Eventual Consistency
Best For	ACID, Complex Queries, Transactions	High Scale, Big Data, High Throughput

⑤ API

- ✓ **API**: Application Programming Interface.
- ✓ **REST API**: Stateless, uses HTTP methods.
- ✓ **Methods**: GET, POST, PUT, DELETE, PATCH
- ✓ **Format**: JSON / XML
- ✓ **Status Codes**: 200, 201, 400, 401, 404, 500



⑥ Cache (Redis)

- ✓ In-memory data store
- ✓ Very fast read/write
- ✓ Supports: Strings, Hashes, Lists, Sets, Sorted Sets
- ✓ Use Cases: Session store, Counters, Leaderboards, Rate limiting



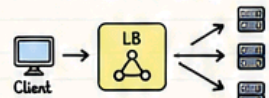
⑦ CDN

- ✓ Content Delivery Network
- ✓ Caches static content at edge locations
- ✓ Reduces latency & server load
- ✓ Improves availability & performance



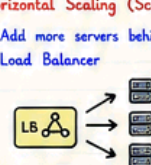
⑧ Load Balancer

- ✓ Distributes traffic across multiple servers
- ✓ Algorithms: Round Robin, Least Connections, IP Hash
- ✓ Performs Health Checks
- ✓ Acts as Reverse Proxy



⑨ Scaling

- Vertical Scaling (Scale Up)**
- Add more power to a single server (CPU, RAM, Storage)
- Horizontal Scaling (Scale Out)**
- Add more servers behind Load Balancer



⑩ Monolith vs Microservices

Monolith

- ✓ Simple to develop & deploy
- ✗ Hard to scale
- ✗ One failure can bring down entire system
- ✗ Tech stack is uniform



Microservices

- ✓ Highly scalable
- ✓ Independent deployment
- ✓ Fault isolation
- ✗ Complex architecture
- ✗ Network overhead



⑪ Quick Interview Tips

- ★ Always ask clarifying questions.
- ★ Start with High-Level Design.
- ★ Consider scale, reliability, and cost early.
- ★ Discuss trade-offs honestly.
- ★ Communicate & justify decisions.



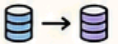
Part 2 - Advanced System Design (18 Slides)

Focus: Build highly scalable systems.

① Recap & Key Concepts



② Database Replication



③ Database Sharding



④ Message Queues (Kafka / RabbitMQ)



⑤ Event-Driven Architecture



⑥ Distributed Caching (Redis)



⑦ Consistent Hashing



⑧ Rate Limiting



⑨ Authentication & Authorization (JWT/OAuth)



⑩ High Availability & Fault Tolerance



⑪ Distributed Transactions (Saga Pattern)



⑫ API Gateway & Service Discovery



⑬ Monitoring, Logging & Tracing



⑭ Disaster Recovery & Multi-Region Deployment



⑮ Design Patterns (CQRS, Circuit Breaker, Bulkhead)



⑯ Design a URL Shortener



⑰ Design WhatsApp / Instagram Feed (Interview Workflow)



⑱ Advanced Cheat Sheet & Interview Tips



Remember

Great designs are not about perfect technology, but about making the right trade-offs for the given scale and requirements.